

AER1217 Lab 4: Stereo Camera-based Visual Odometry

Yuhang Wang 1005048832
Andres Cervera Rozo 1005469217
Mohsen Diraneyya 1006462604

April 5, 2024

1 Introduction

In this lab, we used the stereo camera images from the Kitty dataset to perform visual odometry and estimate the vehicle's 6-D pose relative to its starting pose.

The starter code in the lab handout already had keypoint detection and matching using Scale Invariant Feature Transform (SIFT). After getting these key points that match between the right and left cameras of the current frame and the previous frame, we can use the stereo camera model to estimate the 3D position of each key point.

We implemented the point cloud alignment algorithm, which estimates the vehicle's change in pose between these two frames by aligning the point clouds we got from the matched key points. We also used the RANSAC algorithm to reject the outlier key points and estimate the pose change based only on the inliers.

All the code is implemented in Python and included in the handout, with proper comments and a readme file.

2 Point Cloud Alignment

2.1 Key Point Detection and Feature Matching

In the context of our stereo camera-based visual odometry (VO) system, key point detection and feature matching are crucial processes that's predefined in our starter code. The detector, Scale-Invariant Feature Transform (SIFT), detects key points by searching over all scales and image locations, which were typically found in areas with significant gradient changes. Each keypoint descriptor is a 128-element vector and captures the local gradient patterns around the point. The "find_feature_correspondences" function uses brute-force matching where descriptor from one frame (current left) is compared with descriptors from the other (current right, previous left, previous right) to find the best match based on the some distance calculation. It then outputs $N \times 8$ feature correspondence matrix that represent each correlated features' location within previous and current stereo images.

2.2 Inverse Stereo Camera Model

Equation 1, 2 and Figure 1 shows the stereo camera model where a feature point in the real world represented in the stereo camera center frame can be used to compute the feature point's pixel location in the stereo images. Since we know the feature point pixel location from feature matching, we can use the inverse stereo model, Equation 3, to compute the point's location in the camera center frame. Point cloud of feature points from frame to frame can then be used to determine the transformation between two frames with Scalar-Weighted Point Cloud Alignment.

$$\mathbf{M} = \begin{bmatrix} f_u & 0 & c_u & \frac{f_u b}{2} \\ 0 & f_v & c_v & 0 \\ f_u & 0 & c_u & -\frac{f_u b}{2} \\ 0 & f_v & c_v & 0 \end{bmatrix} \quad (1)$$

$$\begin{bmatrix} u_e \\ v_e \\ u_r \\ v_r \end{bmatrix} = \mathbf{M} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad (2)$$

$$\boldsymbol{\rho} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \mathbf{g}(\mathbf{y}) = \frac{b}{u_l - u_r} \begin{bmatrix} \frac{1}{2}(u_l + u_r) - c_u \\ \frac{f_u}{f_v} \left(\frac{1}{2}(v_l + v_r) - c_v \right) \\ f_u \end{bmatrix} \quad (3)$$

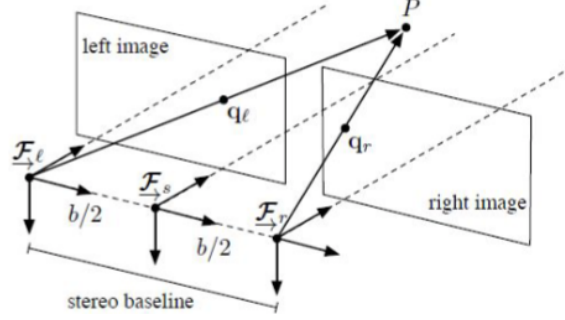


Figure 1: Stereo Camera Model

2.3 Singular Value Decomposition

We used Singular Value Decomposition (SVD) for point cloud alignment to align the point clouds of each two consecutive frames. This is then accumulated through all the frames, hence, odomeStry.

- Define centroid and weights of point clouds as shown in Equation 4, where the weight w , is usually 1.
- Compute outer product matrix (cross covariance matrix between the point clouds) as shown in Equation 5.
- Use SVD on $\mathbf{W}_{ba}$ to compute rotation matrix associated with that transformation as shown in Equation 6.
- Compute translation and full transformation as shown in Equation 7.

$$\mathbf{P}_a = \frac{1}{w} \sum_{j=1}^J w^j \mathbf{P}_a^j, \quad \mathbf{P}_b = \frac{1}{w} \sum_{j=1}^J w^j \mathbf{P}_b^j, \quad w = \sum_{j=1}^J w^j \quad (4)$$

$$\mathbf{W}_{ba} = \frac{1}{w} \sum_{j=1}^J w^j \left(\mathbf{p}_b^{(j)} - \mathbf{p}_b \right) \left(\mathbf{p}_a^{(j)} - \mathbf{p}_a \right)^T \quad (5)$$

$$\mathbf{V} \mathbf{S} \mathbf{U}^T = \mathbf{W}_{ba}, \quad \mathbf{C}_{ba} = \mathbf{V} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(\mathbf{U}) \det(\mathbf{V}) \end{bmatrix} \mathbf{U}^T \quad (6)$$

$$\mathbf{r}_a^{ba} = -\mathbf{C}_{ba}^T \mathbf{p}_b + \mathbf{p}_a, \quad \mathbf{T}_{ba} = \begin{bmatrix} \mathbf{C}_{ba} & -\mathbf{C}_{ba} \mathbf{r}_a^{ba} \\ \mathbf{0} & 1 \end{bmatrix} \quad (7)$$

Note: The general approach for point cloud alignment requires iterations of correlating target with source point cloud and pose estimation with those newly correlated point cloud to generate incremental transformations. The accumulation of incremental transformation is then the final transformation from one frame to the next. The team implemented this method initially, but later found that correlating point cloud is unnecessary since the feature output of "find_feature_correspondences" was already correlated. This was visualized in the third image of Figure 2. The transformation between point cloud is small and the number of points are the same. This also means one iteration of pose estimation is sufficient for each frame.

3 RANSAC

In order to accurately model the camera pose transformation between frames, matching features from the previous and current time frame must be used to derive such transformation. Considering the amount of features that are detected in each time frame, it is unlikely that any singular transformation matrix will perfectly describe each feature’s change in location in 3-dimensional space. As such, it is crucial to only consider inlier features that will improve the accuracy of the overall transformation matrix prediction, and discard any potential outliers that would skew that matrix and introduce error in the reprojection of features from one frame to another.

The RANSAC algorithm is utilized in the script to filter out these outlier features, and reduce any potential error in the visual odometry that these outlier features may introduce. After the corresponding features in both time frames are passed through the stereo inverse model, the resulting features are represented as coordinates in 3D space, relative to the camera. These will be the inputs to the RANSAC algorithm.

To define a transformation between 2 frames, a minimum of 3 coordinate feature pairs are required. For a given iteration of RANSAC, 3 randomly chosen feature pairs are picked out. Through Single Value Decomposition (SVD), as discussed in detail in Section 2.3, the resulting transformation required to transition from the previous to current frame for those particular features can be computed. This transformation can then be applied to all 3D points in the previous frame, and the result is a prediction of where the corresponding features would be in the current time frame.

The predictions of the corresponding locations in the current time frame can then be deemed as inliers or outliers based on their Euclidean distances to the true location of the features in the current time frame. Based on a tuned threshold distance value of 0.35m, all of the predictions can be deemed as either inliers if they closely match the true location, and outliers if they are farther away (potentially due to moving features like vehicles and bikes, or incorrectly matched features output from SIFT). This is the full process for computing 1 iteration of RANSAC.

To gain a more confident prediction of the inliers and outliers, multiple iterations of RANSAC are applied, and only the iteration that yields the greatest number of inliers is considered as the true definition of inliers and outliers moving forward. The number of iterations can be determined through Equation 8, where N is the number of iterations required, p represents the probability that at least one of the random samples drawn will be free of outliers, epsilon is the percentage of outliers in the data, and s is the number of random points required to conduct 1 iteration (3 in this case). As per convention, the value for p was set to 0.99. The value for epsilon was tuned to 0.3, resulting in an N value of 11 iterations. As such, the RANSAC model was iterated 11 times, and the result that yielded the greatest number of inlier features was considered the true classification of feature inliers and outliers. The features deemed to be inliers were then passed to the SVD process to determine a frame transformation between the previous and the current image, to be used for tracking the overall visual odometry.

$$N = \frac{\log(1 - p)}{\log(1 - (1 - \epsilon)^s)} \quad (8)$$

4 Results

Figure 2 shows a sample point cloud obtained from the 10th frame. Here are the point clouds in their two respective time frames, as well as the inlier points in the two respective frames. Figure 3 shows matched and filtered key points on the stereo images. Overall, our RANSAC implementation was robust.

VO faces challenges such as drift accumulation, dependency on distinct environmental features, susceptibility to dynamic elements and varying lighting conditions. These issues can lead to inaccuracies over time, especially in the absence of external localization systems like GPS. The sample data provided is fairly stable, without many motion blur, which is a significant factor to our accurate estimation shown in Figure 4. In this lab, we had to tune RANSAC threshold to obtain good results, Figure 4 shows how our result evolve. This parameter optimized in this case may not perform well in other scenarios. Furthermore, computational constraints often pose a bottleneck, particularly for real-time applications requiring high-resolution data. To enhance odometry performance, integration with other sensors (e.g. wheel encoders) may correct the odometry estimation. Additionally, the adoption of state estimation algorithms such as Kalman filter can

enhance trajectory estimation even in more complicated environments.

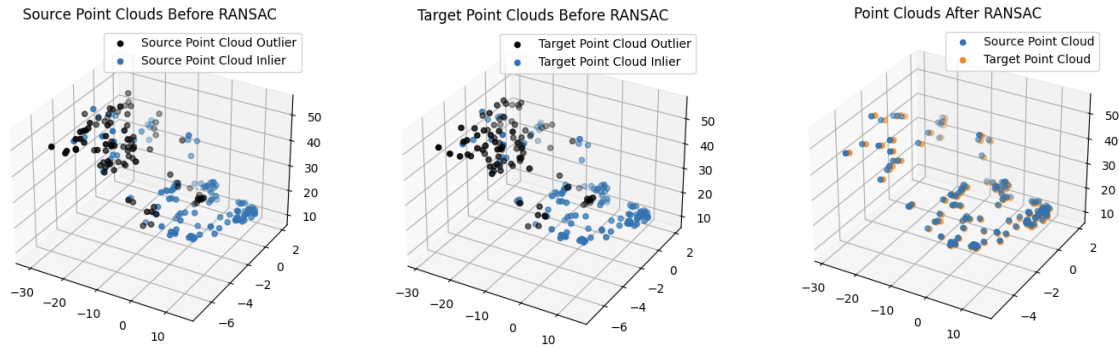


Figure 2: Point Cloud Visualization



Figure 3: Matched Keypoints on the left camera (top) and filtered matched keypoints after RANSAC on the right camera (bottom)

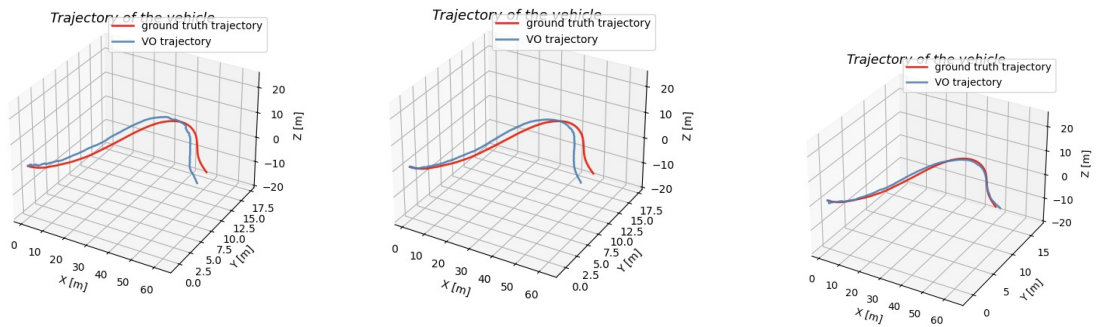


Figure 4: Output vs Ground truth Trajectories

Appendix A: Visual Odometry

```
"""
2021-02 -- Wenda Zhao, Miller Tang

This is the class for a stereo visual Odometry designed
for the course AER 1217H, Development of Autonomous UAS
https://carre.utoronto.ca/aer1217
"""

import math

import cv2 as cv
import numpy as np
from scipy.spatial.transform import Rotation as ScipyRotation

STAGE_FIRST_FRAME = 0
STAGE_SECOND_FRAME = 1
STAGE_DEFAULT_FRAME = 2

# np.random.rand(1217) In starter code, probably typo
np.random.seed(1217)

class StereoCamera:
    def __init__(self, baseline, focal_length, fx, fy, cx, cy):
        self.baseline = baseline
        self.f_len = focal_length
        self.fx = fx
        self.fy = fy
        self.cx = cx
        self.cy = cy

    def find_feature_correspondences(
        kp_l_prev, des_l_prev, kp_r_prev, des_r_prev, kp_l, des_l, kp_r, des_r
    ):
        vertical_px_buffer = 1 # buffer for the epi-polar constraint in number of pixels
        far_thresh = 7 # 7 pixels are approximately 55m away from the camera
        close_thresh = 65 # 65 pixels are approximately 4.2m away from the camera

        number_of_features = len(kp_l)
        bf = cv.BFMatcher(
            cv.NORM_L2, crossCheck=True
        ) # BFMatcher for SIFT or SURF features matching

        # using the current left image as the anchor image
        match_l_r = bf.match(des_l, des_r) # current left to current right
        match_l_l_prev = bf.match(des_l, des_l_prev) # cur left to prev. left
        match_l_r_prev = bf.match(des_l, des_r_prev) # cur left to prev. right

        kp_query_idx_l_r = [mat.queryIdx for mat in match_l_r]
        kp_query_idx_l_l_prev = [mat.queryIdx for mat in match_l_l_prev]
        kp_query_idx_l_r_prev = [mat.queryIdx for mat in match_l_r_prev]

        kp_train_idx_l_r = [mat.trainIdx for mat in match_l_r]
        kp_train_idx_l_l_prev = [mat.trainIdx for mat in match_l_l_prev]
        kp_train_idx_l_r_prev = [mat.trainIdx for mat in match_l_r_prev]

        # loop through all the matched features to find common features
        features_corr = np.zeros((1, 8))
        for pt_idx in np.arange(number_of_features):
            if (
```

```

        (pt_idx in set(kp_query_idx_l_r))
        and (pt_idx in set(kp_query_idx_l_l_prev))
        and (pt_idx in set(kp_query_idx_l_r_prev))
    ):
        temp_feature = np.zeros((1, 8))
        temp_feature[:, 0:2] = kp_l_prev[
            kp_train_idx_l_l_prev[kp_query_idx_l_l_prev.index(pt_idx)]
        ].pt
        temp_feature[:, 2:4] = kp_r_prev[
            kp_train_idx_l_r_prev[kp_query_idx_l_r_prev.index(pt_idx)]
        ].pt
        temp_feature[:, 4:6] = kp_l[pt_idx].pt
        temp_feature[:, 6:8] = kp_r[
            kp_train_idx_l_r[kp_query_idx_l_r.index(pt_idx)]
        ].pt
        features_corr = np.vstack((features_corr, temp_feature))
    features_corr = np.delete(features_corr, 0, axis=0)

    # additional filter to refine feature correspondences
    # 1. drop those features do NOT follow the epi-polar constraint
    features_corr = features_corr[
        (np.absolute(features_corr[:, 1] - features_corr[:, 3]) < vertical_px_buffer)
        & (np.absolute(features_corr[:, 5] - features_corr[:, 7]) < vertical_px_buffer)
    ]

    # 2. drop those features that are either too close or too far from the cameras
    features_corr = features_corr[
        (np.absolute(features_corr[:, 0] - features_corr[:, 2]) > far_thresh)
        & (np.absolute(features_corr[:, 0] - features_corr[:, 2]) < close_thresh)
    ]

    features_corr = features_corr[
        (np.absolute(features_corr[:, 4] - features_corr[:, 6]) > far_thresh)
        & (np.absolute(features_corr[:, 4] - features_corr[:, 6]) < close_thresh)
    ]
    # features_corr:
    # prev_l_x, prev_l_y, prev_r_x, prev_r_y, cur_l_x, cur_l_y, cur_r_x, cur_r_y
    return features_corr

def estimate_pose(corresponding_pair_source, corresponding_pair_target):

    # 1. Compute the centroid of the point clouds without weights
    source_centroid = np.mean(corresponding_pair_source, axis=0)
    target_centroid = np.mean(corresponding_pair_target, axis=0)
    corresponding_pair_source = corresponding_pair_source - source_centroid
    corresponding_pair_target = corresponding_pair_target - target_centroid
    # 2. Compute the outer product (covariance matrix)
    covariance_matrix = np.sum(
        np.array(
            [
                np.matmul(
                    corresponding_pair_source[i].reshape(3, 1),
                    corresponding_pair_target[i].reshape(1, 3),
                )
                for i in range(len(corresponding_pair_source))
            ]
        ),
        axis=0,
    )

    # 3. Using Singular Value Decomposition (SVD)
    u, s, vt = np.linalg.svd(covariance_matrix)

```

```

v = np.transpose(vt)

# 4. Compose final rotation and translation
sign_identity = np.eye(3)
sign_identity[2, 2] = np.sign(np.linalg.det(u) * np.linalg.det(vt))
rotation_matrix_target_source = np.matmul(
    v, np.matmul(sign_identity, np.transpose(u))
)
translation_vector_target_source_target = (
    -np.matmul(np.transpose(rotation_matrix_target_source), target_centroid)
    + source_centroid
)
translation_matrix_target_source = np.identity(4)
translation_matrix_target_source[0:3, 0:3] = rotation_matrix_target_source
translation_matrix_target_source[0:3, 3] = -np.matmul(
    rotation_matrix_target_source, translation_vector_target_source_target
)

return translation_matrix_target_source

def point_cloud_alignment_matrix(
    source_point_cloud, target_point_cloud, number_of_iterations=1
):
    """
    ...

    Parameters:
    source_point_cloud : numpy.ndarray size: (?, 3) either (M x 3) or (3 x 3)
    target_point_cloud : numpy.ndarray size: (?, 3) either (M x 3) or (3 x 3)
    number_of_iterations : number of iterations
    ...

    Returns:
    transform_matrix : numpy.ndarray size: (4, 4)
    ...
    """
    pose = np.identity(4)
    updated_point_cloud = source_point_cloud

    for i in range(number_of_iterations):
        pose_translation_matrix = estimate_pose(updated_point_cloud, target_point_cloud)
        updated_point_cloud_reshaped = np.vstack(
            [np.transpose(updated_point_cloud), np.ones(len(updated_point_cloud))]
        )
        updated_point_cloud = np.matmul(
            pose_translation_matrix, updated_point_cloud_reshaped
        )
        updated_point_cloud = np.transpose(updated_point_cloud[0:3, :])
        pose = np.matmul(pose_translation_matrix, pose)

    return pose

def feature_tracking(prev_kp, cur_kp, img, color=(0, 255, 0), alpha=0.5):
    img = cv.cvtColor(img, cv.COLOR_GRAY2BGR)
    cover = np.zeros_like(img)
    # Draw the feature tracking
    for i, (new, old) in enumerate(zip(cur_kp, prev_kp)):
        a, b = new.ravel()
        c, d = old.ravel()
        a, b, c, d = int(a), int(b), int(c), int(d)
        cover = cv.line(cover, (a, b), (c, d), color, 2)

```



```

        cover = cv.circle(cover, (a, b), 3, color, -1)
    frame = cv.addWeighted(cover, alpha, img, 0.75, 0)

    return frame

def compute_max_iter_ransac(certainty):
    """
    Use the RANSAC eqn to determine a max number of iterations to perform to gain a
    degree of certainty in
    inlier detection. Max_iter = frac{log(1 - p)}{log(1 - (1-e)^s)}, where p is the
    certainty, e
    is the estimated proportion of outliers, and s is the number of samples needed to
    estimate the model.

    Parameters:
    certainty : float
    value from 0 to 1 (non-inclusive) of how certain we want to be. Usually set to 0.99
    or higher

    Returns:
    int
    The number of iterations required to achieve the desired certainty
    """
    pts_picked = 3
    perc_outlier = 0.3
    number_of_iterations = math.log(1 - certainty) / math.log(
        1 - (1 - perc_outlier) ** pts_picked
    )
    return math.ceil(number_of_iterations)

def calculate_euclid(p_cur, p_transformed):
    """
    Determine the Euclidean distance of each feature between the current frame, and the
    transformation
    applied to the previous frame.

    Parameters:
    p_cur          : numpy.ndarray size: (N, 3)
    p_transformed  : numpy.ndarray size: (N, 3)
    2 supposedly similar frames of features.

    Returns:
    numpy.ndarray size: (N, 1)
    Each element represents the Euclidean distance for that row between the 2 frames
    """
    squared_diff = (p_cur - p_transformed) ** 2
    sum_squared_diff = np.sum(squared_diff, axis=1)
    dist = np.sqrt(sum_squared_diff)
    return dist

def filter_inliers_ransac(p_before, p_cur, max_iter):
    """
    Apply RANSAC inlier filtering to the given data to only preserve the features that
    are inliers.

    Parameters:
    p_before      : numpy.ndarray size: (number_of_pairs, 3)
    p_cur         : numpy.ndarray size: (number_of_pairs, 3)
    max_iter      : int

```


The np arrays are each the unfiltered 3D point clouds of the features from the previous and current frame.

Max_iter is the number of iterations of RANSAC required to achieve a certain confidence level (0.99).

Returns:

```
p_a      : numpy.ndarray size: (M, 3)
p_b      : numpy.ndarray size: (M, 3)
inlier_indices : numpy.ndarray size: (M, 1);
```

Info:

number_of_pairs >= M. The returned arrays are the 3D point clouds of the inlier features for the previous and current frames, respectively. The elements of inlier_indices represent the indices of number_of_pairs that were considered inliers.

```
"""

max_inliers = -1
top_t = np.zeros((4, 4))
inlier_threshold_euclidean = 0.35
high_inlier_perc_threshold = 0.95 # more than the high_inlier_perc_threshold percent
    of points are inliers, can break RANSAC
number_of_pairs = p_before.shape[0]

def inliers_from_t(transform_matrix, pairs_before, pairs_current):
    """
    Given a certain transform transform_matrix, apply that transform to the previous
    frame point cloud and determine
    the inliers by comparing the Euclidean distance to the current frame point cloud.

    Parameters:
    transform_matrix      : numpy.ndarray size: (4, 4)
    p_before              : numpy.ndarray size: (number_of_pairs, 3)
    p_cur                 : numpy.ndarray size: (number_of_pairs, 3)

    The np arrays are each the unfiltered 3D point clouds of the features from the
    previous and current frame.

    Returns:
    inliers_cnt      : int          value <= number_of_pairs
    inlier_indices   : numpy.ndarray size: (number_of_inliers, 1), and each element is
        value from 0-> number_of_pairs-1

    Returns the number of inliers found, as well as the indices of those inliers
    w.r.t. The original
    number_of_pairs features.
    """
    p_before_transform = np.hstack(
        (pairs_before, np.ones((number_of_pairs, 1))))
    ).T # 4 x number_of_pairs where the last row is 1s

    p_after_transform = (
        transform_matrix @ p_before_transform
    ) # 4x4 @ 4 x number_of_pairs => 4 x number_of_pairs
    p_transformed = p_after_transform[:3, :].T # number_of_pairs x 3 X, Y, Z

    dist = calculate_euclid(pairs_current, p_transformed) # number_of_pairs,
    mask = dist < inlier_threshold_euclidean # Boolean len number_of_pairs
    this_inlier_indices = np.where(mask)[0] # M,
    this_number_of_inliers = np.count_nonzero(mask) # int

    return this_number_of_inliers, this_inlier_indices
```

```

def one_iter_ransac(pairs_before, pairs_current):
    """
    Run a single iteration of RANSAC using three random feature pairs from the
    previous and current frames.

    Parameters:
    p_before : numpy.ndarray size: (number_of_pairs, 3)
    p_cur : numpy.ndarray size: (number_of_pairs, 3)
    The np arrays are each the unfiltered 3D point clouds of the features from the
    previous and current frame.

    Returns:
    iteration_transform_matrix : numpy.ndarray size: (4, 4)
    number_of_inliers : int value <= number_of_pairs

    Returns the transform applied to the previous frame, as well as the number of
    inliers found
    as a result of the transformation.
    """
    rand_idx = np.random.choice(
        number_of_pairs, 3, replace=False
    ) # 3 random samples
    prev_test_points = pairs_before[rand_idx]
    cur_test_points = pairs_current[rand_idx]
    this_temp_transform_matrix = point_cloud_alignment_matrix(
        prev_test_points, cur_test_points
    )
    this_number_of_inliers, _ = inliers_from_t(
        this_temp_transform_matrix, pairs_before, pairs_current
    )
    return this_temp_transform_matrix, this_number_of_inliers

print(f"starting ransac - maximum number of iteration number: {max_iter}")
for i in range(max_iter):
    iteration_transform_matrix, number_of_inliers = one_iter_ransac(p_before, p_cur)
    if number_of_inliers > max_inliers:
        max_inliers = number_of_inliers
        top_t = iteration_transform_matrix
        if (number_of_inliers / number_of_pairs) > high_inlier_perc_threshold:
            print(f"finished ransac with {i} iterations, inliers > 95%")
            break
    if i == (max_iter - 1):
        print(f"finished ransac after reaching the maximum number of iterations")

_, inlier_indices = inliers_from_t(top_t, p_before, p_cur) # repeat best transform

p_a = p_before[inlier_indices]
p_b = p_cur[inlier_indices]

return p_a, p_b, inlier_indices

class VisualOdometry:
    def __init__(self, cam):
        self.frame_stage = 0
        self.cam = cam
        self.new_frame_left = None
        self.last_frame_left = None
        self.new_frame_right = None
        self.last_frame_right = None
        self.C = np.eye(3) # current rotation (initiated to be eye matrix)
        self.r = np.zeros((3, 1)) # current translation (initiated to be zeros)
        self.kp_l_prev = None # previous key points (left)

```

```

self.des_l_prev = None # previous descriptor for key points (left)
self.kp_r_prev = None # previous key points (right)
self.des_r_prev = None # previous descriptor key points (right)
self.detector = cv.SIFT_create() # using sift for detection
self.feature_color = (255, 191, 0)
self.inlier_color = (32, 165, 218)

def feature_detection(self, img):
    kp, des = self.detector.detectAndCompute(img, None)
    feature_image = cv.drawKeypoints(img, kp, None)
    return kp, des, feature_image

def convert_features_to_3d(self, features_corr):
    """
    Take the u and v pixel information (x and y) of the left and right camera from the
    previous and current
    frames and produce the 3D point cloud points for the previous frame and current
    frame, independently.

    Parameters:
    features_corr : list
    The features_corr is of size N x 8 where each row is the detected feature and the
    columns follow
    the following order:
    [prev_l_x, prev_l_y, prev_r_x, prev_r_y, cur_l_x, cur_l_y, cur_r_x, cur_r_y]

    Returns:
    p_before: numpy.ndarray
    p_cur: numpy.ndarray
    Each array is of size N x 3 and represents the 3D coordinates of each detected
    feature, where each array
    represents the respective timeframe.
    """

    # assuming epi-polar from starter code, only look at x coords
    f_prev = features_corr[:, :4] # N x 4
    f_cur = features_corr[:, 4:8] # N x 4

    def inverse_stereo(
        stereo_camera_object, l_and_r_feats
    ): # input N x 4 (l_u, l_v, r_u, r_l)
        """
        Convert features from left and right camera to 3d points using the inverse
        stereo model

        Parameters:
        features_corr : list
        The features_corr is of size N x 4 where each row is the detected feature and
        the columns follow
        the following order:
        [l_u, l_v, r_u, r_v]
        Returns:
        numpy.ndarray
        An array of size N x 3 representing the 3D coordinates of the features
        """
        l_u = l_and_r_feats[:, 0]
        disparity = (
            l_u - l_and_r_feats[:, 2]
        ) # assumes epi-polar from starter code filtering and rectification, l_u - r_u
        z = np.array(
            stereo_camera_object.cam.baseline
            * stereo_camera_object.cam.f_len
            / disparity

```

```

    ) # N,
    x = np.array(
        z * (l_u - stereo_camera_object.cam.cu) / stereo_camera_object.cam.f_len
    ) # N,
    y = np.array(
        z
        * (l_and_r_feats[:, 1] - stereo_camera_object.cam.cv)
        / stereo_camera_object.cam.f_len
    ) # N,
    points_3d = np.column_stack((x, y, z))
    return points_3d # N x 3 (x, y, z)

p_before = inverse_stereo(self, f_prev)
p_cur = inverse_stereo(self, f_cur)
return p_before, p_cur # each N x 3

# POSE ESTIMATION #####

def pose_estimation(self, features_corr):
    """
    Process the detected features of the left and right camera in the previous and
    current frames as 3D
    point clouds, identify the features that are inliers, and use those features to
    compute the rotation
    and translation from the previous frame to the current frame.

    Parameters:
    features_corr : list
    The features_corr is of size N x 8 where each row is the detected feature and the
    columns follow
    the following order:
        [prev_l_x, prev_l_y, prev_r_x, prev_r_y, cur_l_x, cur_l_y, cur_r_x, cur_r_y]

    Returns:
    rotation_matrix : numpy.ndarray size: (3, 3)
    r : numpy.ndarray size: (3, 1)
    f_r_prev : list size: (M, 2)
    f_r_cur : list size: (M, 2)

    Rotation_matrix and r are the translations applied to the previous frame to
    convert them to the current_frame.
    F_r_prev are the filtered features for the right camera in the previous frame, and
    f_r_cur are
    the filtered features for the right camera in the current frame.
    """

    # ----- start your code here ----- #

    # 1. Convert to 3d cloud points, two sets: before: [x, y, z], and current: [x, y,
    # z],
    # 2. Iteratively run RANSAC to get inliers by fitting rotation_matrix and r to 3
    # points,
    # 3. Choose a version that had the most inliers,
    # 4. Use M features for cloud alignment
    p_before, p_cur = self.convert_features_to_3d(
        features_corr
    ) # np arrays N x 3 each
    certainty = 0.99
    max_iter = compute_max_iter_ransac(certainty)
    p_a, p_b, inliers_indices = filter_inliers_ransac(
        p_before, p_cur, max_iter
    ) # M x 3 arrays

```

```

transform_matrix = point_cloud_alignment_matrix(p_a, p_b)
rotation_matrix = transform_matrix[:3, :3]
r = transform_matrix[:3, 3]

filtered_features_corr = features_corr[inliers_indices]
f_r_prev, f_r_cur = (
    filtered_features_corr[:, 2:4],
    filtered_features_corr[:, 6:8],
)
return rotation_matrix, r, f_r_prev, f_r_cur

def process_first_frame(self, img_left, img_right):
    kp_l, des_l, feature_l_img = self.feature_detection(img_left)
    kp_r, des_r, feature_r_img = self.feature_detection(img_right)

    self.kp_l_prev = kp_l
    self.des_l_prev = des_l
    self.kp_r_prev = kp_r
    self.des_r_prev = des_r

    self.frame_stage = STAGE_SECOND_FRAME
    return img_left, img_right

def process_second_frame(self, img_left, img_right):
    kp_l, des_l, feature_l_img = self.feature_detection(img_left)
    kp_r, des_r, feature_r_img = self.feature_detection(img_right)

    # compute feature correspondence
    features_corr = find_feature_correspondences(
        self.kp_l_prev,
        self.des_l_prev,
        self.kp_r_prev,
        self.des_r_prev,
        kp_l,
        des_l,
        kp_r,
        des_r,
    ) # Nx8
    # draw the feature tracking on the left img
    img_l_tracking = feature_tracking(
        features_corr[:, 0:2],
        features_corr[:, 4:6],
        img_left,
        color=self.feature_color,
    )

    # lab4 assignment: compute the vehicle pose
    [self.C, self.r, f_r_prev, f_r_cur] = self.pose_estimation(features_corr)

    # draw the feature (inliers) tracking on the right img
    img_r_tracking = feature_tracking(
        f_r_prev, f_r_cur, img_right, color=self.inlier_color, alpha=1.0
    )

    # update the key point features on both images
    self.kp_l_prev = kp_l
    self.des_l_prev = des_l
    self.kp_r_prev = kp_r
    self.des_r_prev = des_r
    self.frame_stage = STAGE_DEFAULT_FRAME

    return img_l_tracking, img_r_tracking

```

```

def process_frame(self, img_left, img_right):
    kp_l, des_l, feature_l_img = self.feature_detection(img_left)
    kp_r, des_r, feature_r_img = self.feature_detection(img_right)

    # compute feature correspondence
    features_corr = find_feature_correspondences(
        self.kp_l_prev,
        self.des_l_prev,
        self.kp_r_prev,
        self.des_r_prev,
        kp_l,
        des_l,
        kp_r,
        des_r,
    ) # N x 8
    # draw the feature tracking on the left img
    img_l_tracking = feature_tracking(
        features_corr[:, 0:2],
        features_corr[:, 4:6],
        img_left,
        color=self.feature_color,
    )

    # lab4 assignment: compute the vehicle pose
    [self.C, self.r, f_r_prev, f_r_cur] = self.pose_estimation(features_corr)

    # draw the feature (inliers) tracking on the right img
    img_r_tracking = feature_tracking(
        f_r_prev, f_r_cur, img_right, color=self.inlier_color, alpha=1.0
    )

    # update the key point features on both images
    self.kp_l_prev = kp_l
    self.des_l_prev = des_l
    self.kp_r_prev = kp_r
    self.des_r_prev = des_r

    return img_l_tracking, img_r_tracking

def update(self, img_left, img_right):

    self.new_frame_left = img_left
    self.new_frame_right = img_right

    if self.frame_stage == STAGE_DEFAULT_FRAME:
        frame_left, frame_right = self.process_frame(img_left, img_right)

    elif self.frame_stage == STAGE_SECOND_FRAME:
        frame_left, frame_right = self.process_second_frame(img_left, img_right)

    else:
        frame_left, frame_right = self.process_first_frame(img_left, img_right)

    self.last_frame_left = self.new_frame_left
    self.last_frame_right = self.new_frame_right

    return frame_left, frame_right

"""
2021-02 -- Wenda Zhao, Miller Tang.

this is the code base for a stereo visual odometry designed
for the course AER 1217H, Development of Autonomous UAS

```

```

https://carre.utoronto.ca/aer1217.
The Kitti dataset, raw_data/City/2011_09_26_drive_0005, is used in this assignment.
https://www.cvlibs.net/datasets/kitti/raw_data.php
"""

import os

import cv2 as cv
import matplotlib.pyplot as plt
import numpy as np
import scipy.io as sio
from stereo_vo_base import StereoCamera, VisualOdometry

def main():
    # current working directory
    cwd = os.getcwd()

    # the ground_truth_pose.mat is saved from the kitti
    # 'raw_data_development_kit' -> run_demoVehiclePath.m
    # see http://www.cvlibs.net/datasets/kitti/raw_data.php for more details
    oxts_pose_gt = sio.loadmat("ground_truth_pose.mat")["pose"]

    # image sequence
    sequence_num = oxts_pose_gt.shape[1]
    pose_gt = np.zeros((sequence_num, 4, 4))
    # save ground truth into pose_gt
    for i in range(sequence_num):
        pose_gt[i] = oxts_pose_gt[0, i]

    # ----- Parameter ----- #
    path_0 = "/2011_09_26/2011_09_26_drive_0005_sync/image_00/data/"
    path_1 = "/2011_09_26/2011_09_26_drive_0005_sync/image_01/data/"
    zero_num = 10
    image_directory = os.path.abspath(
        os.path.join(os.path.dirname(__file__), "CityData")
    )
    # '/CityData/2011_09_26_calib/calib_cam_to_cam.txt'
    # baseline, focalLength, fx, fy, cu, cv
    cam = StereoCamera(0.537, 721.5377, 721.5377, 721.5377, 609.5593, 172.8540)
    vo = VisualOdometry(cam)

    # global transform to camera (starts as identity)
    transform_matrix = np.eye(4)
    transform_matrix_hist = np.zeros((sequence_num, 4, 4))
    transform_matrix_hist[0] = transform_matrix

    # convert to vehicle frame
    # calibration results provided by kitti dataset in '/CityData/2011_09_26_calib/'
    transform_matrix_imu_to_velo = np.array(
        [
            [9.999976e-01, 7.553071e-04, -2.035826e-03, -8.086759e-01],
            [-7.854027e-04, 9.998898e-01, -1.482298e-02, 3.195559e-01],
            [2.024406e-03, 1.482454e-02, 9.998881e-01, -7.997231e-01],
            [0, 0, 0, 1],
        ]
    )

    transform_matrix_velo_to_cam = np.array(
        [
            [7.533745e-03, -9.999714e-01, -6.166020e-04, -4.069766e-03],
            [1.480249e-02, 7.280733e-04, -9.998902e-01, -7.631618e-02],

```



```

        [9.998621e-01, 7.523790e-03, 1.480755e-02, -2.717806e-01],
        [0, 0, 0, 1],
    ]
)

transform_matrix_cam_to_cam_center = np.array(
    [
        [1.0, 0.0, 0.0, -0.537 / 2.0],
        [0.0, 1.0, 0.0, 0.0],
        [0.0, 0.0, 1.0, 0.0],
        [0.0, 0.0, 0.0, 1.0],
    ]
)

transform_matrix_imu_to_cam = transform_matrix_velo_to_cam.dot(
    transform_matrix_imu_to_velo
)

transform_matrix_imu_to_cam_center = transform_matrix_cam_to_cam_center.dot(
    transform_matrix_imu_to_cam
)

transform_matrix_cam_center_to_imu = np.linalg.inv(
    transform_matrix_imu_to_cam_center
)

# initialization offset: The ground truth data of the vehicle starts with (0, 0, 0)
# in GPS/
# But in the VO estimation, we set the initial position of the stereo-camera's center
# as (0,0,0) using initial r.
# Therefore, we convert the ground truth data to camera center position in GPS frame
# with (0, 0, 0) as the initial position.
# The conversion is done by translation vector t_cv_v.

# translation from vehicle frame to camera center frame expressed in vehicle frame
# (from Kitti website)
t_cv_v = np.array([1.09, -0.32 - 0.537 / 2.0, 0.8])

cam_center_gt = np.zeros((sequence_num, 3))
for idx in range(sequence_num):
    cam_center_gt[idx, :] = pose_gt[idx, 0:3, 3] + t_cv_v

# Transformation in vehicle frame
transform_matrix_vehicle = np.zeros((sequence_num, 4, 4))

# record video
os.chdir(cwd)
write_frames = []

for img_id in range(sequence_num):
    print(f"Start processing image {img_id} / {sequence_num}")
    img_left = cv.imread(
        image_directory + path_0 + str(img_id).zfill(zero_num) + ".png", 0
    )
    img_right = cv.imread(
        image_directory + path_1 + str(img_id).zfill(zero_num) + ".png", 0
    )

    # finite-state machine
    # updates the rotation matrix and r
    frame_left, frame_right = vo.update(img_left, img_right)

    # Create a white margin between two frames

```

```

margin = np.ones_like(frame_left) * 255
vertical_frame = np.concatenate((frame_left, margin[0:25], frame_right), axis=0)

# Update the transformation matrix
transform_matrix_update = np.vstack(
    (
        (np.concatenate((vo.C, vo.r.reshape(-1, 1)), axis=1)),
        np.array([[0, 0, 0, 1]]),
    )
)
transform_matrix = transform_matrix_update.dot(transform_matrix)
# Store the history of transformation matrix
transform_matrix_hist[img_id] = transform_matrix
# convert to vehicle frame
transform_matrix_vehicle[img_id] = transform_matrix_cam_center_to_imu.dot(
    np.linalg.inv(transform_matrix_hist[img_id])
)

cv.imshow("Visual Odometry", vertical_frame)
write_frames.append(vertical_frame)
if cv.waitKey(1) & 0xFF == ord("q"):
    break

print("VO ends\n")
cv.destroyAllWindows()
video = cv.VideoWriter(
    "./video_stereo.avi", cv.VideoWriter_fourcc(*"XVID"), 5.0, (1242, 375)
) # 375*2 + 25 (margin)

for frame in write_frames:
    video.write(frame)
    if cv.waitKey(5) & 0xFF == ord("q"):
        break
video.release()
cv.destroyAllWindows()

# save the estimated transformation matrix
np.save("VO_T.npy", transform_matrix_vehicle)

fig_traj = plt.figure(facecolor="white")
ax_t = fig_traj.add_subplot(111, projection="3d")
ax_t.plot(
    cam_center_gt[:, 0],
    cam_center_gt[:, 1],
    cam_center_gt[:, 2],
    color="red",
    label="ground truth trajectory",
    linewidth=1.9,
    alpha=0.9,
)
ax_t.plot(
    transform_matrix_vehicle[:, 0, 3],
    transform_matrix_vehicle[:, 1, 3],
    transform_matrix_vehicle[:, 2, 3],
    color="steelblue",
    label="VO trajectory",
    linewidth=1.9,
    alpha=0.9,
)
ax_t.set_xlabel(r"$X$ [m]")
ax_t.set_ylabel(r"$Y$ [m]")
ax_t.set_zlabel(r"$Z$ [m]")
ax_t.legend()

```

```
ax_t.set_zlim3d(-20.0, 25.0)
plt.title(
    r"Trajectory of the vehicle",
    fontsize=13,
    fontweight=0,
    color="black",
    style="italic",
    y=1.02,
)
plt.show()

if __name__ == "__main__":
    print("We are using OpenCV version {}".format(cv.__version__))
    main()
```
